Course Project Proposal: Review 1

Course Code: BCSE306L — Artificial Intelligence

Project Title: Multi-Heuristic A^* Campus Navigation and Optimal Route Finder

Team Members: Nimisha(24BCT0188), Rohan(24BCE0985), Ishaan(24BDS0243), Rahul(24BCE3006), Tanishq(24BCE0980)

Date: September 11, 2026

1. Problem Statement

Given a directed graph representing campus landmarks and pedestrian pathways with varying physical edge distances, find the shortest, collision-free pedestrian route between any user-selected origin and destination landmark while minimizing node expansions and search runtime.

2. Module and Technique

- **Syllabus Module:** Module 2 — Problem Solving based on Searching.
- **Core Technique:** A^* Graph Search Algorithm ($f(n) = g(n) + h(n)$) implemented completely from scratch in pure Python using a custom binary min-heap.
- **Comparative Baselines:** Uniform Cost Search ($h(n) = 0$), Greedy Best-First Search ($f(n) = h(n)$), and A^* using distinct heuristic functions.

3. Input and Output

- **Input:**
    - A weighted graph $G = (V, E)$, where each vertex $v \in V$ represents a campus landmark with planar Cartesian coordinates (x, y), and each edge $e \in E$ has an associated physical walking distance $c(u, v)$ in meters.
    - Runtime query: Start_Node and Goal_Node.
- **Output:**
    - An ordered sequence of vertices representing the optimal pedestrian route.
    - Quantitative performance metrics: Total path cost (meters), total nodes expanded, and execution latency (milliseconds).

4. Formal AI Formulation

- **State Space (S):** Discrete campus coordinate locations $V = \{v_1, v_2, \dots, v_n\}$, where state $s \in V$.

- **Initial State (s_0):** Starting landmark $s_{start} \in V$.
- **Actions ($A(s)$):** Traversal along any valid pathway to an adjacent landmark v' where an edge $(s, v') \in E$ exists.
- **Transition Model ($Result(s, a)$):** Deterministic state transition resulting in node v'.
- **Goal Test ($Goal(s)$):** Evaluates True if $s == s_{goal}$, and False otherwise.
- **Path Cost Function ($g(n)$):** Cumulative distance traversed from the initial state:

$$g(v') = g(s) + c(s, v')$$

- **Heuristic Functions ($h(n)$):**
    1. *Euclidean Distance (Straight-line displacement; admissible and consistent)*:

$$h_1(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

    2. *Manhattan Distance (Grid-based displacement)*:

$$h_2(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$$

5. Experimental Comparison Plan

Benchmarking will be conducted across 50 randomized source-destination pairs on the campus topology:

- **Algorithms Compared:** Uniform Cost Search vs. Greedy Best-First Search vs. A^* Search.
- **Heuristics Compared:** A^* with Euclidean (h_1) vs. A^* with Manhattan (h_2) vs. Zero Heuristic ($h = 0$).
- **Metrics:** Path cost optimality (meters), search efficiency (total nodes expanded), and runtime latency (ms).

6. Team Role Division

- **Nimisha:** Implementation of the core A^* graph search engine and frontier/explored-set state tracking.
- **Rohan:** Custom data structure development, including the zero-dependency binary min-heap priority queue and sift operations.
- **Ishaan:** Baseline search algorithm implementations, specifically Uniform Cost Search (UCS) and Greedy Best-First Search.
- **Rahul:** Mathematical formulation, coding of heuristic functions (h_1 Euclidean, h_2 Manhattan), and proof of heuristic admissibility.

- **Tanishq:** Campus graph modelling (planar coordinate extraction, edge distance weights) and automated batch benchmarking runner.